BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Computer Vision

2024-25 First Semester, M.Tech (AIML)

Session #6:

Harris Corner Detector, HoG & Applications

Raghavendra Singh

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

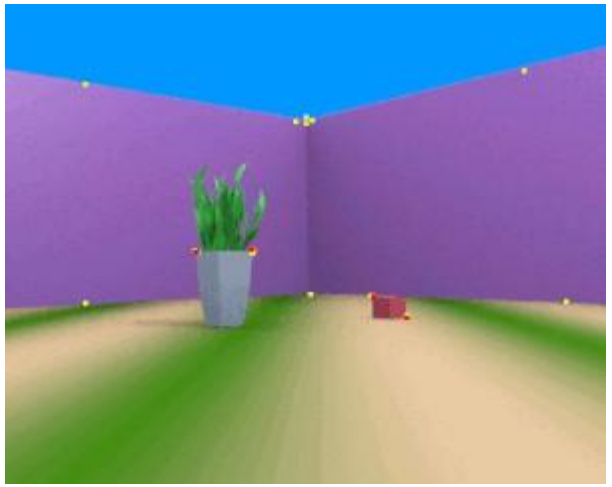
- Harris Corner detector
- Histogram of Oriented Gradients

Adopted from (1) Rick Szeliski's lecture notes, CSE576, Spring 05 (2) CS5670: Computer Vision - Local features & Harris corner detection

Corner Detection

What is Corner Detection? Corner detection is a fundamental technique in computer vision used to identify points in an image where there is a significant change in both the horizontal and vertical directions. These points, known as **corners**, typically occur at the intersection of two edges and represent distinct, stable features in an image.

Why Corners Matter? Corners are important because they are highly distinctive, invariant under scaling, rotation, and illumination changes, making them robust for tasks like object recognition, image matching, and motion tracking.



https://en.wikipedia.org/wiki/Corner_detection

Applications of Corner Detection

Object Recognition: Corners provide unique, stable features that help in identifying and recognizing objects within an image. These features are used in object recognition algorithms, particularly in applications like facial recognition, logo detection, and tracking specific items.

Image Matching: Corners are used for matching images in different conditions, such as rotation, scaling, or changes in viewpoint. They are helpful in tasks like panorama stitching, where different images are aligned to form a larger image.

Feature Tracking: In video or motion analysis, corner points are tracked over time to follow the movement of objects or parts of an image. This is used in optical flow estimation, motion tracking, and dynamic scene analysis.

Camera Calibration: Corner detection is used in the calibration of cameras, where the 2D positions of known 3D points (like the corners of a checkerboard) are used to compute the intrinsic and extrinsic parameters of the camera.

3D Reconstruction: In stereo vision, corners are crucial for estimating depth information from multiple views. Correspondences between corner points in different views are used to reconstruct the 3D geometry of objects.

Robotics and Navigation: Robots use corner detection for feature-based localization and mapping, as well as obstacle detection and avoidance. By identifying corners in the environment, a robot can navigate or map its surroundings effectively.

Augmented Reality (AR): In AR applications, corner detection helps track physical objects or surfaces in real-time to overlay digital content accurately. For example, detecting corners on a table to place virtual objects in a scene.

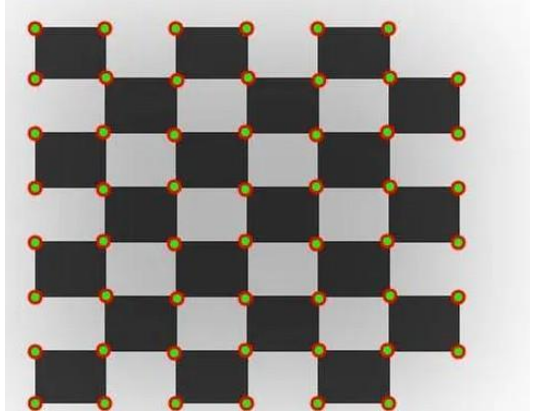
Document Analysis and OCR: In optical character recognition (OCR), corners of text blocks or documents help segment the text and identify the structure of the page, improving text extraction accuracy.

Video Compression: Corner detection helps in compressing video data by focusing on the movement of key features (like corners) rather than entire frames, reducing redundancy in video encoding.

Motion Analysis: In sports or surveillance, corner detection can be used to track significant changes in object motion, such as identifying player movements or vehicles in traffic analysis.

Key Techniques for Corner Detection

Common corner detection algorithms include the **Harris Corner Detector** and the **Shi-Tomasi Corner Detector**, which evaluate the local image structure to identify these critical points.

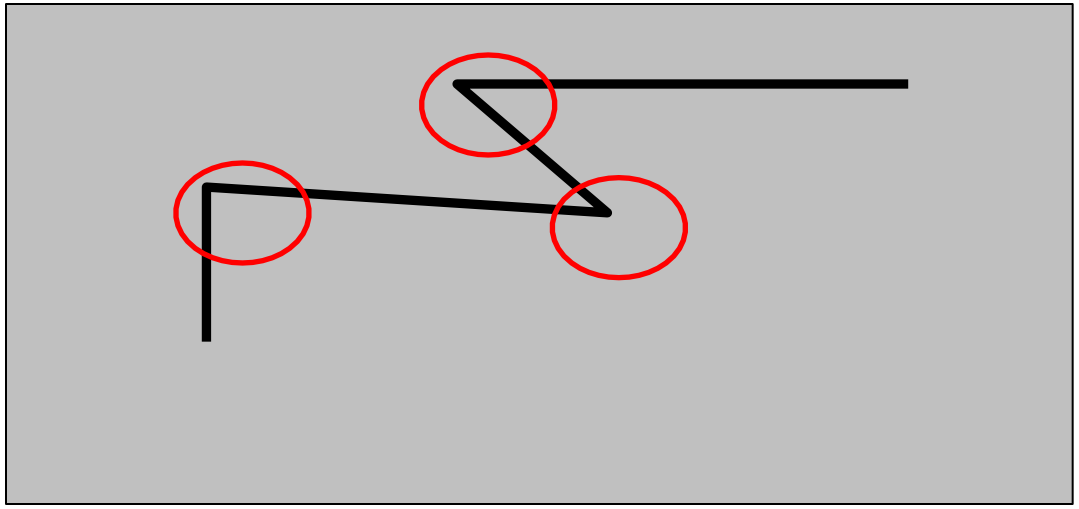


https://medium.com/@deepanshut041/introduction-to-harris-corner-detector-32a88850b3f6

This technique forms the backbone for numerous applications across fields such as robotics, augmented reality, and image processing, enhancing the accuracy and efficiency of visual systems.

Harris corner detector

- C.Harris, M.Stephens. “A Combined Corner and Edge Detector”. 1988



This slide introduces the **Harris Corner Detector**, a foundational algorithm in computer vision used to identify "interesting" points in an image—specifically corners.

Here is a breakdown of the key elements on the slide:

1. The Core Reference

The text cites the seminal 1988 paper, **"A Combined Corner and Edge Detector"** by Chris Harris and Mike Stephens. This paper improved upon earlier methods (like the Moravec corner detector) by making the detection more sensitive to corners and less sensitive to noise.

2. The Visual Intuition

The image at the bottom illustrates what the algorithm is looking for.

- **The Black Lines:** Represent edges or boundaries in an image.
- **The Red Circles:** Highlight the **corners**.

In computer vision, a "corner" is defined as a point where there is a large variation in intensity in **all directions**.

3. How It Works (The "Big Idea")

While not detailed in text on this specific slide, the Harris method relies on a mathematical concept called the **Second Moment Matrix** (or Harris Matrix). It looks at a small window around a pixel and asks: *"If I shift this window slightly in any direction, does the intensity change significantly?"*

Feature Type	Change in Intensity when Shifted
Flat Region	No change in any direction.
Edge	Change in one direction, but no change along the edge.
Corner	**Significant change in all directions.**

The Math Behind the Detection

The algorithm calculates a score R for each window. If R is large and positive, it's a corner. The formula typically involves the eigenvalues (λ_1, λ_2) of the local structure tensor:

$$R = \det(M) - k(\text{trace}(M))^2$$

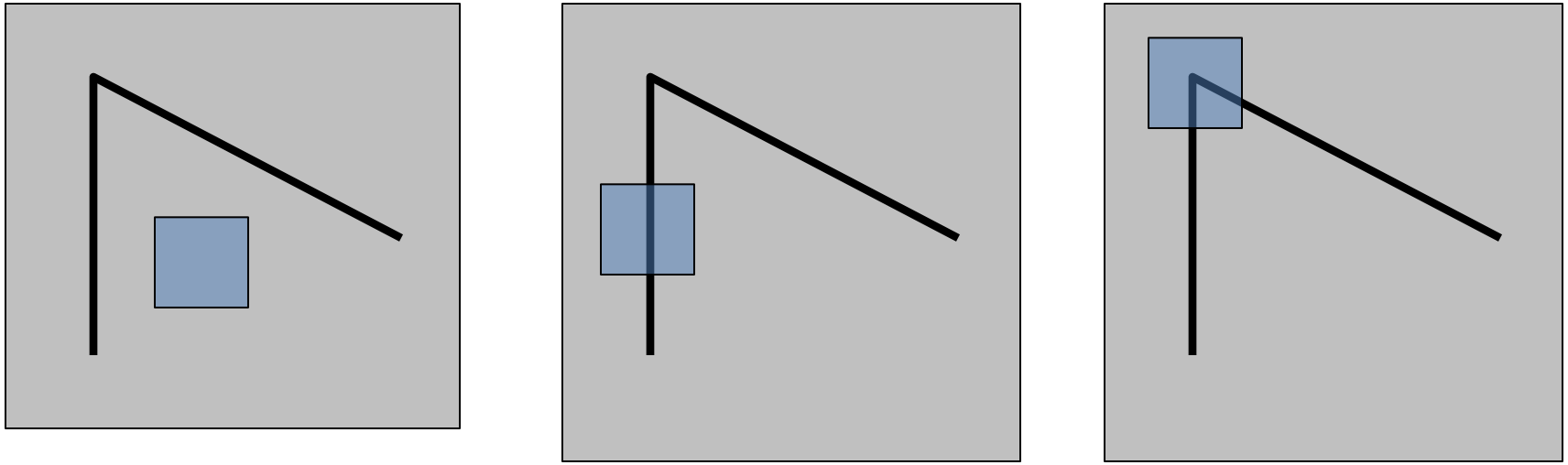
Why is this important?

Corners are excellent "features" because they are **stable**. If you rotate an image or change the lighting slightly, the corners usually stay in the same place, making them perfect for tasks like **image stitching** (panoramas), **object tracking**, and **3D reconstruction**.

Local measures of uniqueness

Suppose we only consider a small window of pixels

– What defines whether a feature is a good or bad candidate?



Credit: S. Seitz, D. Frolova, D. Simakov

This slide is a classic introduction to **Computer Vision**, specifically addressing the concept of **Feature Detection** (like how your phone's camera recognizes a face or how a panorama is stitched together).

The core question it asks is: **"How can a computer tell if a specific small patch of an image is unique enough to be tracked?"**

To explain this, the slide uses three scenarios involving a "small window" (the blue square):

1. The Flat Region (Left Image)

- **The Scene:** The blue window is over a solid, uniform background.
- **The Problem:** If you move the window slightly in any direction, the pixels inside look exactly the same.
- **Verdict:** This is a **bad candidate**. There is no "uniqueness," making it impossible to localize or track precisely.

2. The Edge (Middle Image)

- **The Scene:** The blue window is placed over a straight line (an edge).
- **The Problem:** While you can tell if the window moves *across* the line, you can't tell if it slides *along* the line. The pixels look identical as long as the window stays on that edge. This is known as the **Aperture Problem**.
- **Verdict:** This is a **mediocre candidate**. It's better than a flat area, but still ambiguous in one dimension.

3. The Corner (Right Image)

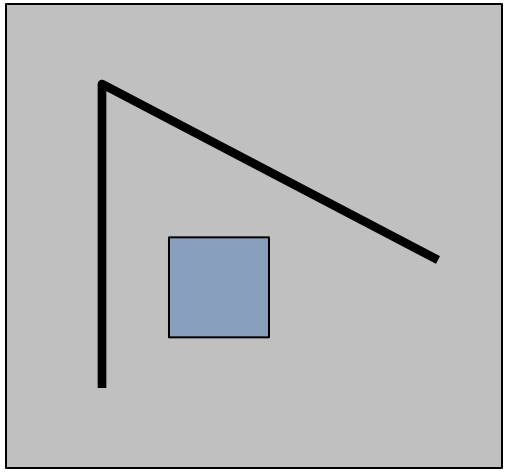
- **The Scene:** The blue window is placed over the vertex where two lines meet.
- **The Solution:** If you move the window in **any direction** (up, down, left, right, or diagonally), the pattern of pixels inside the window changes significantly.
- **Verdict:** This is a **good candidate**. Corners are highly "unique" local measures, which is why algorithms like the **Harris Corner Detector** specifically look for these points to use as "features" or "landmarks."

Summary Table

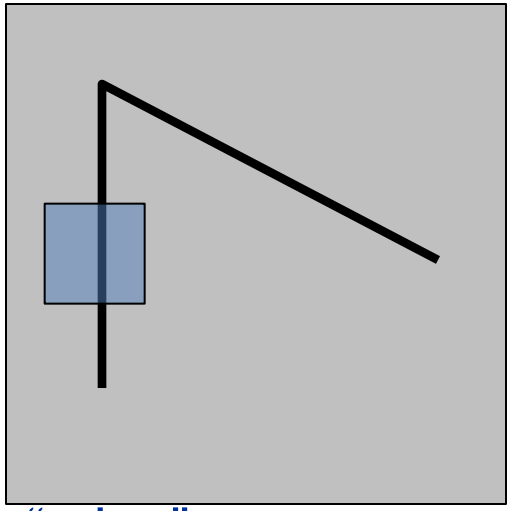
Feature Type	Gradient Change	Tracking Quality
Flat	No change in any direction	Poor
Edge	Change in one direction only	Fair (Ambiguous)
Corner	Change in all directions	**Excellent**

Local measures of uniqueness

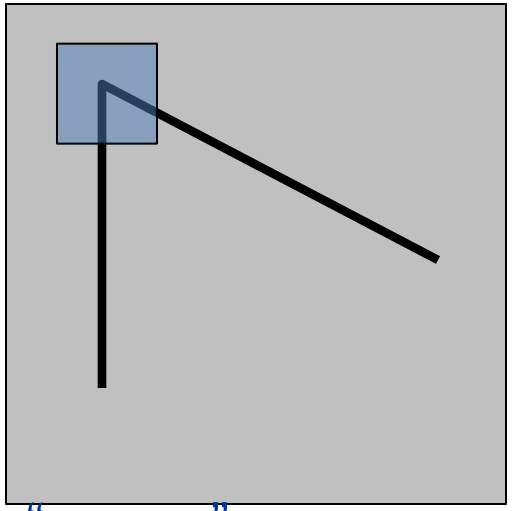
- How does the window change when you shift it?
- Shifting the window in any direction causes a big change



"flat" region:
no change in all directions



"edge":
no change along the edge direction



"corner":
significant change in all directions

Credit: S. Seitz, D. Frolova, D. Simakov

This image illustrates the fundamental intuition behind **Harris Corner Detection**, a classic technique in computer vision used to identify distinct features in an image.

The core idea is to see how much the pixel intensities within a small window change when you shift that window slightly in any direction (u, v).

Key Regions Explained

Region	Behavior upon Shifting	Uniqueness Level
"Flat" region	No matter which way you move the window, the pixels look basically the same.	**Low**: These are common and hard to track.
"Edge" region	Moving the window *along* the edge results in no change, but moving *across* the edge causes a big change.	**Medium**: Good for some tasks, but suffer from the "aperture problem."
"Corner" region	Moving the window in **any** direction results in a significant change in pixel intensity.	**High**: These are unique points that are easy to track across different images.

The Mathematical Intuition

To turn this visual logic into code, we use the **Sum of Squared Differences (SSD)**. We look for a window where the following error function $E(u, v)$ is high for all shifts:

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u, y+v) - I(x,y)]^2$$

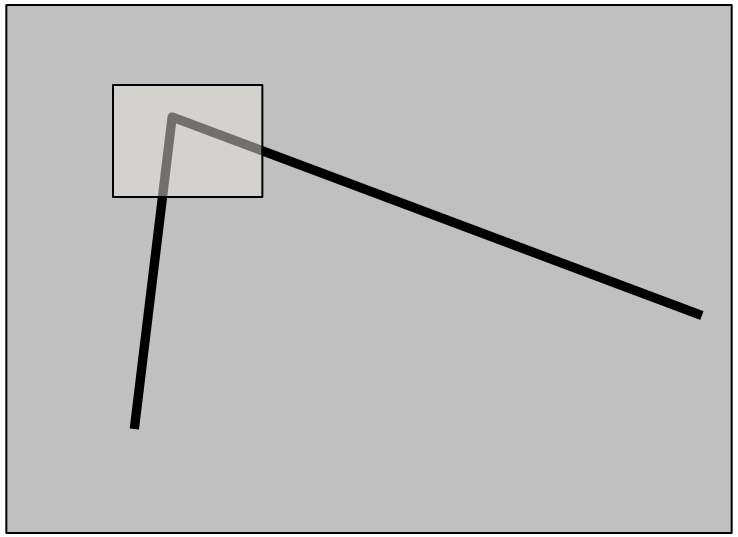
- $w(x, y)$: The window function (usually Gaussian).
- $I(x, y)$: The intensity at a specific pixel.
- $I(x + u, y + v)$: The intensity after the shift.

By using a Taylor expansion, this simplifies into a matrix M (often called the **Second Moment Matrix**). The eigenvalues (λ_1, λ_2) of this matrix tell us what kind of region we are looking at:

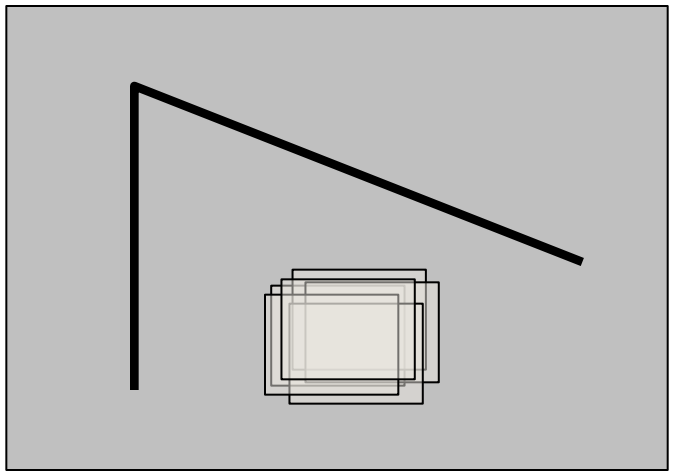
- **Both λ are small:** Flat region.
- **One λ is large, one is small:** Edge.
- **Both λ are large:** Corner!

The Basic Idea

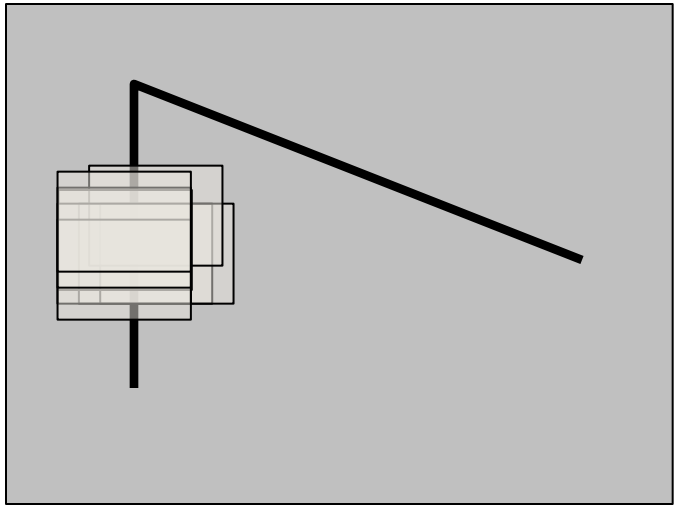
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give *a large change* in intensity



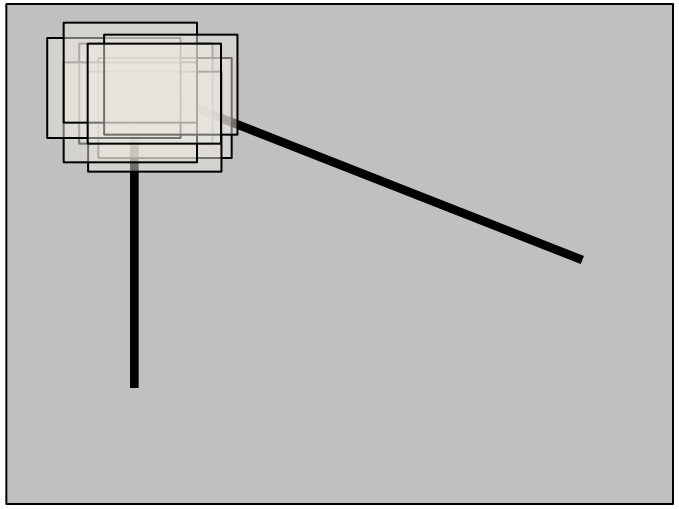
Harris Detector: Basic Idea



"flat" region:
no change in all directions



"edge":
no change along the edge direction



"corner":
significant change in all directions

Harris Detector: Mathematics

Change of intensity for the shift $[u,v]$:

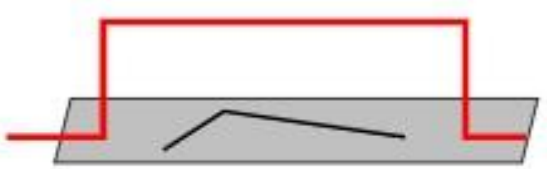
$$E(u,v) = \sum_{x,y} w(x,y)\left[I(x+u,y+v) - I(x,y)\right]^2$$

Window function

Shifted intensity

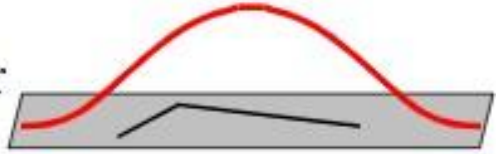
Intensity

Window function $w(x,y)$ =



1 in window, 0 outside

or



Gaussian

Harris Detector: Mathematics

Taylor Series expansion of *I*:

$$I(x+u, y+v) = I(x,y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \text{higher order terms}$$

If the motion (u,v) is small, then first order approximation is good

$$I(x+u, y+v) \approx I(x,y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v$$

$$\approx I(x,y) + \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

shorthand: $I_x = \frac{\partial I}{\partial x}$

Plugging this into the formula on the previous slide...

This slide explains the mathematical foundation of the **Harris Corner Detector**, specifically focusing on how it approximates changes in intensity when you move a small "window" over an image.

Here is the breakdown of the concepts shown:

1. The Goal: Measuring Intensity Change

To find a corner, we need to know how much the image intensity I changes when we shift our focus by a tiny amount (u, v). If the intensity changes significantly in all directions, we've likely found a corner.

2. Taylor Series Expansion

Since images are essentially 2D functions of intensity, we use a **Taylor Series expansion** to predict the value of a pixel at a new location $(x+u, y+v)$ based on its current location (x, y).

The slide shows the multi-variable Taylor expansion:

$$I(x+u, y+v) = I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \text{higher order terms}$$

- $I(x, y)$: The original intensity.
- $\frac{\partial I}{\partial x}u$: The change in intensity in the x-direction.
- $\frac{\partial I}{\partial y}v$: The change in intensity in the y-direction.

Why does this matter?

This approximation is the "secret sauce" that allows the Harris Detector to be fast. Instead of re-calculating the entire image intensity for every possible shift, it uses the **gradients (I_x, I_y)**—which can be calculated easily using simple filters (like Sobel operators)—to estimate how the image looks when shifted.

3. First-Order Approximation

In computer vision, we usually assume the shift (u, v) is very small. This allows us to ignore the "higher order terms" (which are mathematically complex and computationally expensive) and use a **linear approximation**:

$$I(x+u, y+v) \approx I(x, y) + \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

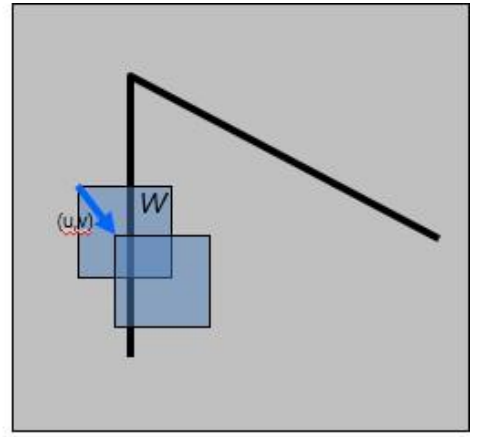
Key Shorthand:

- I_x: The partial derivative of the image with respect to x (the horizontal gradient).
- I_y: The partial derivative of the image with respect to y (the vertical gradient).
- $\begin{bmatrix} I_x & I_y \end{bmatrix}$: This is the **gradient vector**, representing the direction of steepest increase in intensity.

Harris Detector: Mathematics

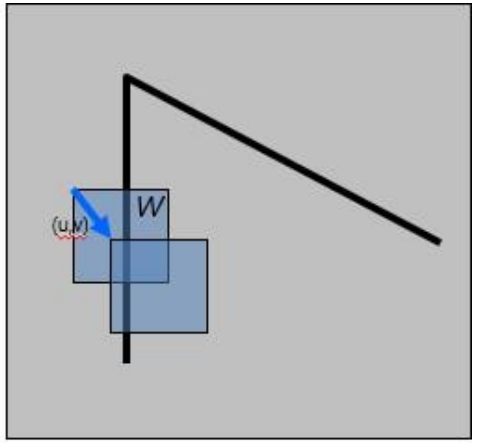
Consider shifting the window *W* by (*u*,*v*)

- define an SSD "error" *E*(*u*,*v*):



$$
\begin{aligned}
E(u,v) &= \sum_{(x,y)\in W} [I(x+u,y+v) - I(x,y)]^2 \\
&\approx \sum_{(x,y)\in W} [I(x,y) + I_x u + I_y v - I(x,y)]^2 \\
&\approx \sum_{(x,y)\in W} [I_x u + I_y v]^2
\end{aligned}
$$

Harris Detector: Mathematics



Consider shifting the window *W* by (*u*,*v*)

- define an SSD "error" *E*(*u*,*v*):

$$E(u,v) \approx \sum_{(x,y)\in W} [I_x u + I_y v]^2$$

$$\approx Au^2 + 2Buv + Cv^2$$

$$A = \sum_{(x,y)\in W} I_x^2 \qquad B = \sum_{(x,y)\in W} I_x I_y \qquad C = \sum_{(x,y)\in W} I_y^2$$

- Thus, *E*(*u*,*v*) is locally approximated as a quadratic error function

Harris Detector: Mathematics

The surface $E(u,v)$ is locally approximated by a quadratic form.

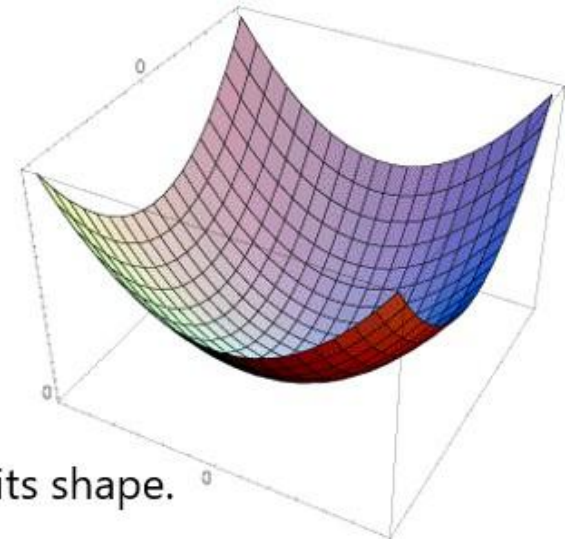
$$E(u,v) \approx Au^2 + 2Buv + Cv^2$$

$$\approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_{H} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$A = \sum_{(x,y)\in W} I_x^2$$

$$B = \sum_{(x,y)\in W} I_x I_y$$

$$C = \sum_{(x,y)\in W} I_y^2$$



Let's try to understand its shape.

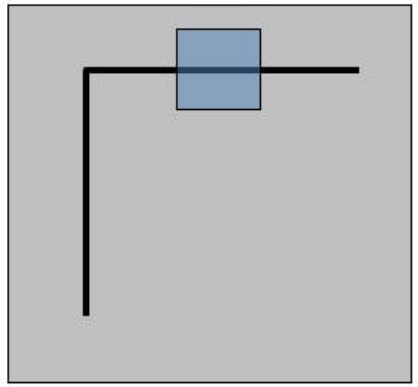
Harris Detector: Mathematics

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_{H} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$A = \sum_{(x,y)\in W} I_x^2$$

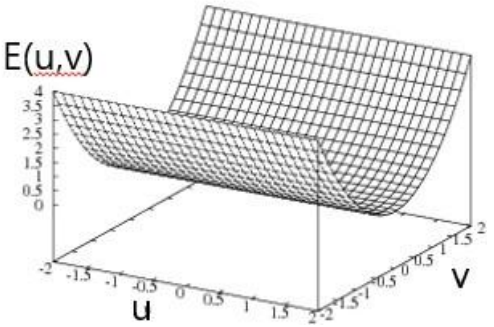
$$B = \sum_{(x,y)\in W} I_x I_y$$

$$C = \sum_{(x,y)\in W} I_y^2$$



Horizontal edge: $I_x = 0$

$$H = \begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix}$$



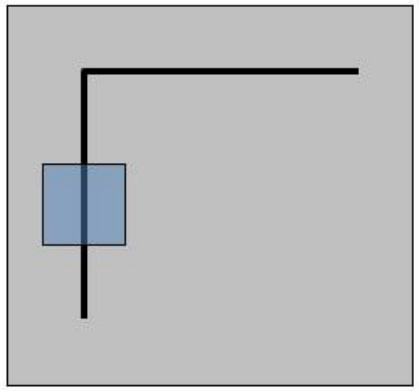
Harris Detector: Mathematics

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_{H} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$A = \sum_{(x,y)\in W} I_x^2$$

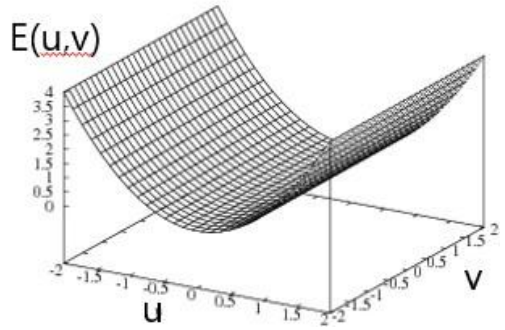
$$B = \sum_{(x,y)\in W} I_x I_y$$

$$C = \sum_{(x,y)\in W} I_y^2$$



Vertical edge: $I_y = 0$

$$H = \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix}$$



The **Harris Corner Detector** is a fundamental algorithm in computer vision used to identify "interesting" points in an image, such as corners or edges. The goal of the mathematics you've provided is to determine how much the image intensity changes when we shift a small window over a specific point.

Here is a breakdown of the mathematical derivation shown in your images:

---

1. The Sum of Squared Differences (SSD)

The process begins by defining an error function $E(u, v)$ that calculates the difference in intensity when a window W is shifted by a small amount (u, v).

$$E(u,v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x,y)]^2$$

- $I(x, y)$: Intensity at a pixel.
- $I(x+u, y+v)$: Intensity at the shifted pixel.
- **If E is high for all shifts**, the window likely contains a **corner**.
- **If E is high for some shifts but low for others**, it's an **edge**.
- **If E is low for all shifts**, it's a **flat region**.

2. Taylor Series Expansion

To make the formula computationally efficient, we use a first-order Taylor expansion to approximate the shifted intensity $I(x+u, y+v)$:

$$I(x+u, y+v) \approx I(x,y) + I_x u + I_y v$$

Where I_x and I_y are the partial derivatives of the image (the gradients in the x and y directions). When we plug this back into the SSD equation, the $I(x, y)$ terms cancel out, leaving:

$$E(u,v) \approx \sum_{(x,y) \in W} [I_x u + I_y v]^2$$

3. The Second Moment Matrix (H)

By expanding the square $(I_x u + I_y v)^2$ and pulling u and v out of the summation, we can rewrite the error function as a **Quadratic Form**:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} H \begin{bmatrix} u \\ v \end{bmatrix}$$

The matrix H (sometimes called the Structure Tensor) is defined as:

$$H = \begin{bmatrix} A & B \\ B & C \end{bmatrix} = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

This matrix H captures the local distribution of image gradients. The "shape" of the error surface $E(u, v)$ tells us what kind of feature we are looking at.

4. Visualizing the Error Surface

The images illustrate three distinct cases based on the gradients within the window:

Feature Type	Gradient Behavior	Matrix H Shape	Error Surface Visualization
Flat Region	$I_x \approx 0, I_y \approx 0$	All entries near 0	A flat plane; no change in any direction.
Horizontal Edge	$I_x \approx 0, I_y > 0$	$\begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix}$	A "trough" or "valley"; error only increases when moving vertically.
Vertical Edge	$I_x > 0, I_y \approx 0$	$\begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix}$	A "trough"; error only increases when moving horizontally.
Corner	Both I_x, I_y are large	High values for A, B, C	A "bowl" shape; error increases sharply in **all** directions.

5. Summary of the Goal

In practice, we don't calculate the full surface for every pixel. Instead, we look at the **eigenvalues** (λ_1, λ_2) of matrix H:

- If both eigenvalues are small, it's a **flat region**.
- If one is large and one is small, it's an **edge**.
- If both are large, we've found a **corner**.

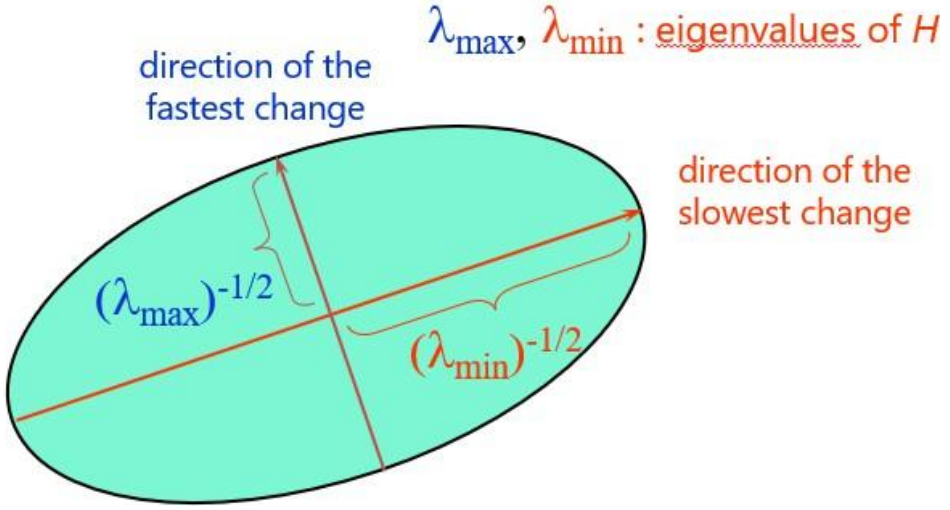
Harris Detector: Mathematics

General case

We can visualize H as an ellipse with axis lengths determined by the *eigen values* of H and orientation determined by the *eigenvectors* of H

Ellipse equation:

$$[u \;\; v] \; H \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$



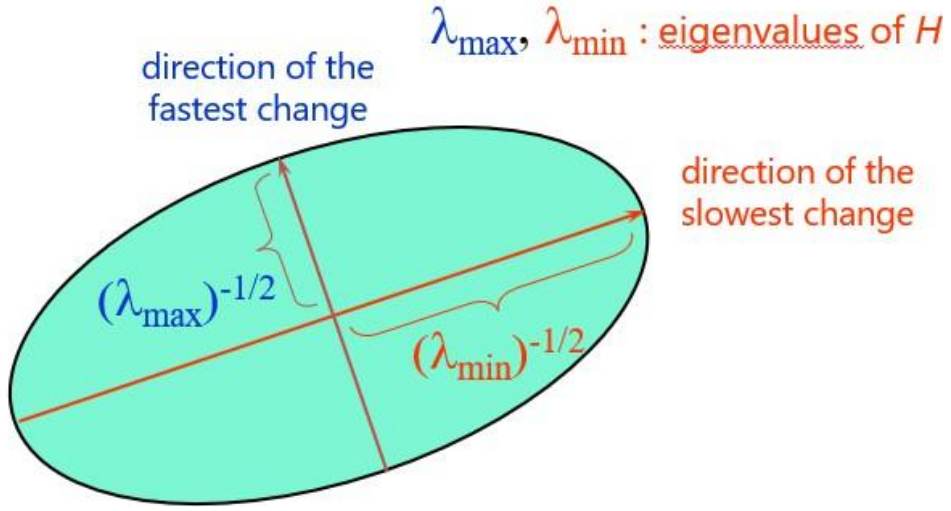
Harris Detector: Mathematics

General case

We can visualize H as an ellipse with axis lengths determined by the *eigen values* of H and orientation determined by the *eigenvectors* of H

Ellipse equation:

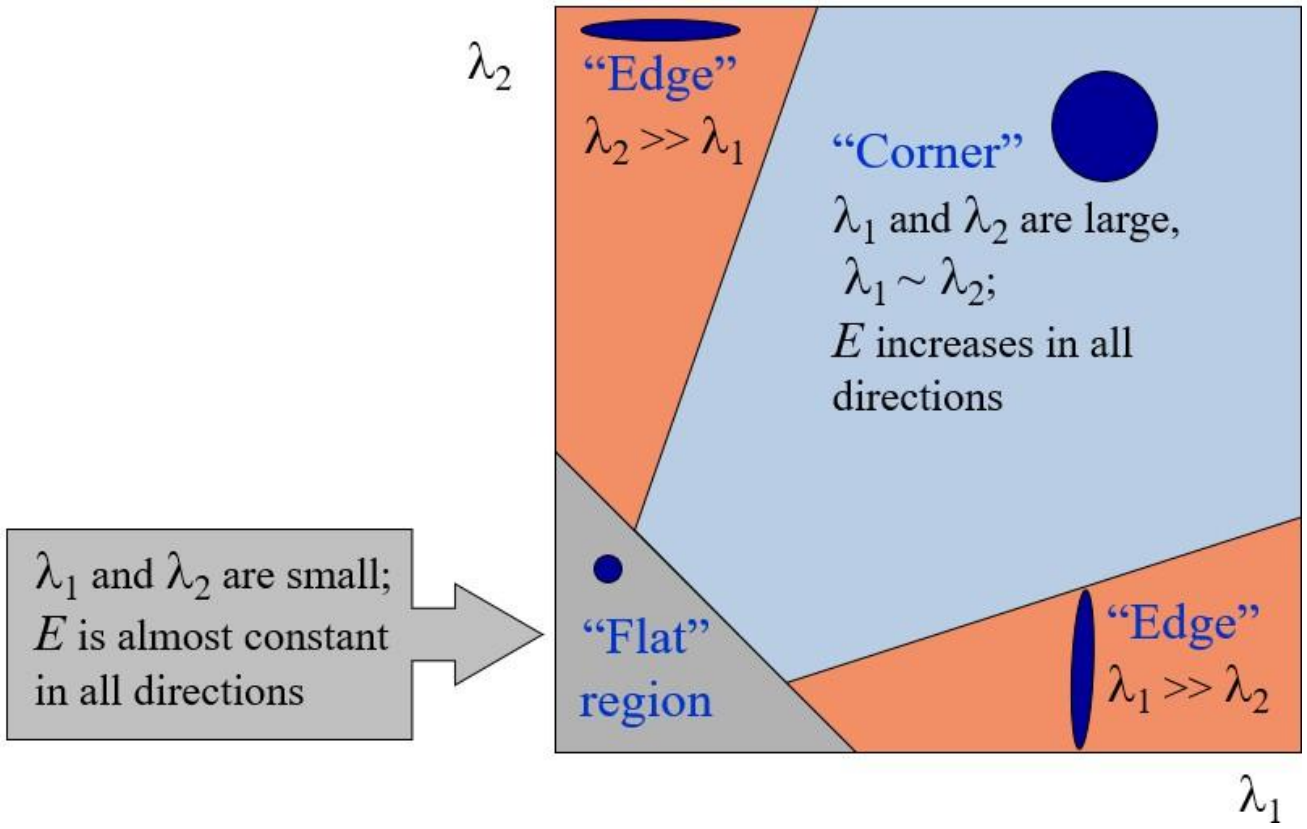
$$[u \;\; v] \; H \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$



Harris Detector: Mathematics

Interpreting the eigenvalues

Classification of image points using eigenvalues of M:



These slides explain the mathematical intuition behind the **Harris Corner Detector**, a fundamental algorithm in computer vision used to identify "interesting" points (like corners) in an image.

The core idea is to look at a small window around a pixel and see how much the intensity changes when you shift that window in any direction.

Slides 1 & 2: The Geometry of Change

The first two slides (which are identical) explain how we model image intensity changes using a matrix, often called the **Second Moment Matrix** or the **Harris Matrix** (H).

- **The Ellipse Visualization:** The local structure of the image can be visualized as an ellipse. This ellipse represents the "uncertainty" or the spread of intensity gradients in a local neighborhood.
- **Eigenvectors:** These define the **orientation** of the ellipse.
  - One eigenvector points in the direction of the fastest change (across an edge).
  - The other points in the direction of the slowest change (along an edge).
- **Eigenvalues ($\lambda_{max}, \lambda_{min}$):** These determine the **lengths** of the ellipse axes.
  - The lengths are proportional to $(\lambda)^{-1/2}$.
  - A **large eigenvalue** means the intensity changes very rapidly in that direction.
  - A **small eigenvalue** means the intensity is relatively constant in that direction.

Slide 3: Interpreting the Eigenvalues

This slide is the "decision-making" part of the math. By looking at the relationship between the two eigenvalues (λ_1 and λ_2), we can classify the local image patch into one of three categories:

1. Flat Regions

- **Condition:** Both λ_1 and λ_2 are very small.
- **Description:** The intensity is nearly constant in all directions. The ellipse would look like a very small, faint circle. There is no significant gradient.

2. Edges

- **Condition:** One eigenvalue is large, and the other is small ($\lambda_1 \gg \lambda_2$ or $\lambda_2 \gg \lambda_1$).
- **Description:** There is a strong intensity change in one direction (across the edge) but very little change in the perpendicular direction (along the edge). The ellipse is very long and skinny.

3. Corners

- **Condition:** Both λ_1 and λ_2 are large and roughly equal.
- **Description:** Shifting the window in **any** direction causes a significant change in intensity. This is the definition of a corner. The ellipse is large and more circular.

Summary Table

Region	λ_1	λ_2	Visualization
Flat	Small	Small	Small Circle
Edge	Large	Small	Long, narrow ellipse
Corner	Large	Large	Large Circle/Fat ellipse

Harris Detector: Mathematics

Corner detection summary

Here's what you do:

- Compute the gradient at each point in the image

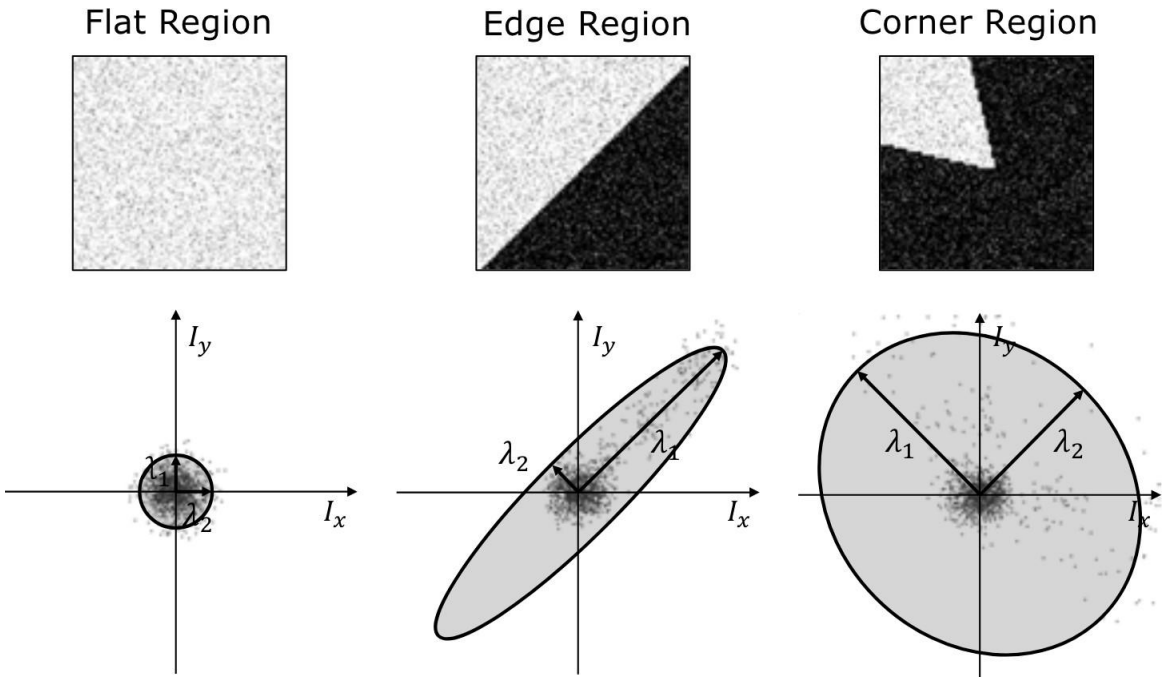
1. **Eigenvalues**:
   - The eigenvalues λ_1 and λ_2 are derived from the structure tensor M at each pixel.
   - The structure tensor is represented as:
     $$M = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$
     where I_x and I_y are the gradients of the image in the x and y directions.
2. **Corner Response Function R**:
   - The **corner response** R is used to identify whether a pixel is part of a corner, an edge, or a flat region:
     $$R = \det(M) - k \cdot (\text{trace}(M))^2$$
     where:
     - $\det(M) = \lambda_1 \cdot \lambda_2$ (product of eigenvalues).
     - $\text{trace}(M) = \lambda_1 + \lambda_2$ (sum of eigenvalues).
     - k is a constant (typically between 0.04 and 0.06).

Finding values

Second Moments for a Region:

$$a = \sum_{i \in W} \left(I_{x_i}{}^2\right) \qquad b = 2\sum_{i \in W} \left(I_{x_i} I_{y_i}\right)$$

$$c = \sum_{i \in W} \left(I_{y_i}{}^2\right)$$ W: Window centered at pixel



Ellipse Axes Lengths:

$$\lambda_1 = E_{max} = \frac{1}{2}\left[a + c + \sqrt{b^2 + (a-c)^2}\right]$$

$$\lambda_2 = E_{min} = \frac{1}{2}\left[a + c - \sqrt{b^2 + (a-c)^2}\right]$$

Harris Detector: Mathematics

Measure of corner response:

$$R = \det M - k\left(\operatorname{trace} M\right)^2$$

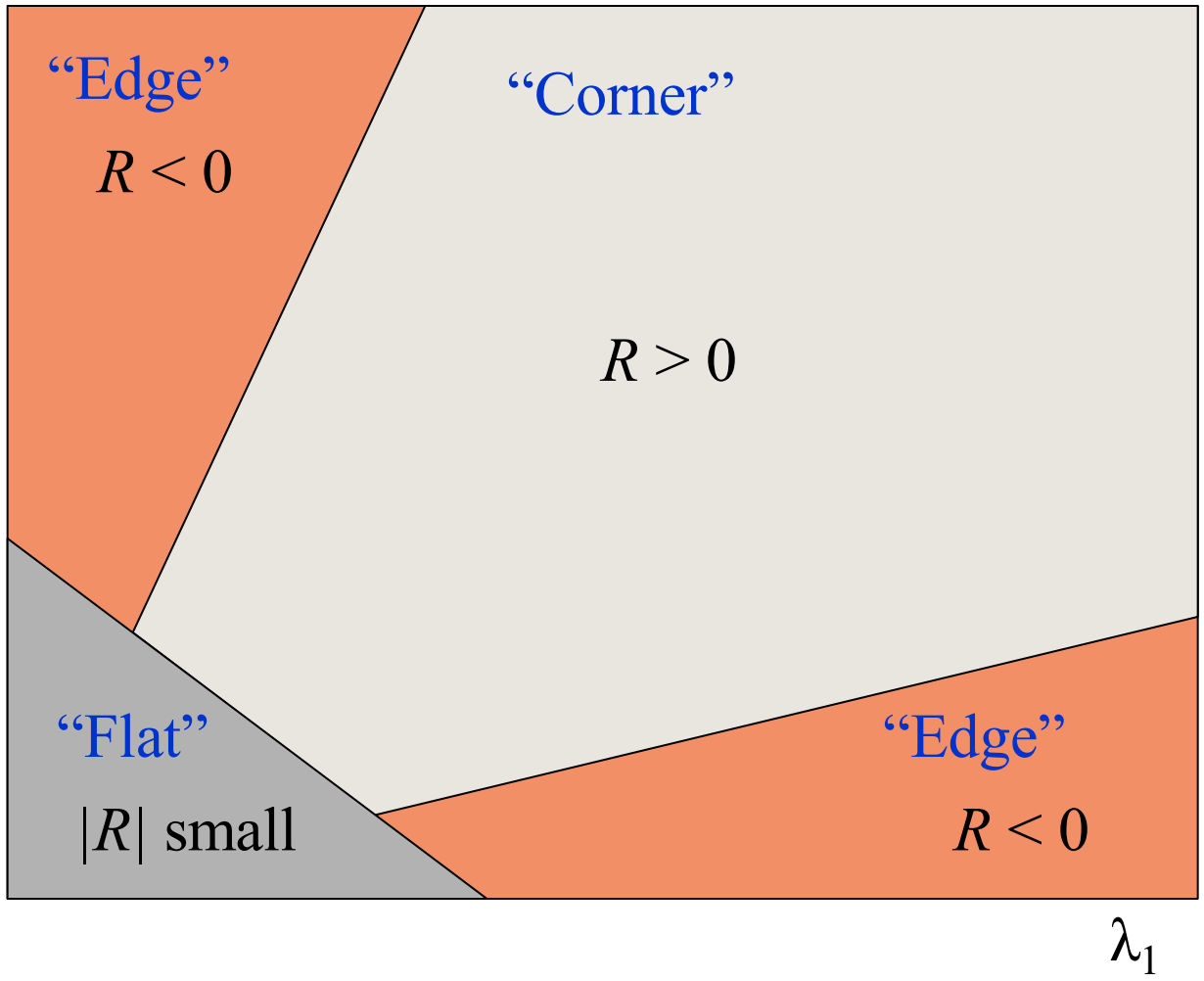
$$\det M = \lambda_1 \lambda_2$$

$$\operatorname{trace} M = \lambda_1 + \lambda_2$$

(k – empirical constant, $k = 0.04\text{-}0.06$)

Harris Detector: Mathematics

- R depends only on eigenvalues of M
- R is large for a corner
- R is negative with large magnitude for an edge
- $|R|$ is small for a flat region



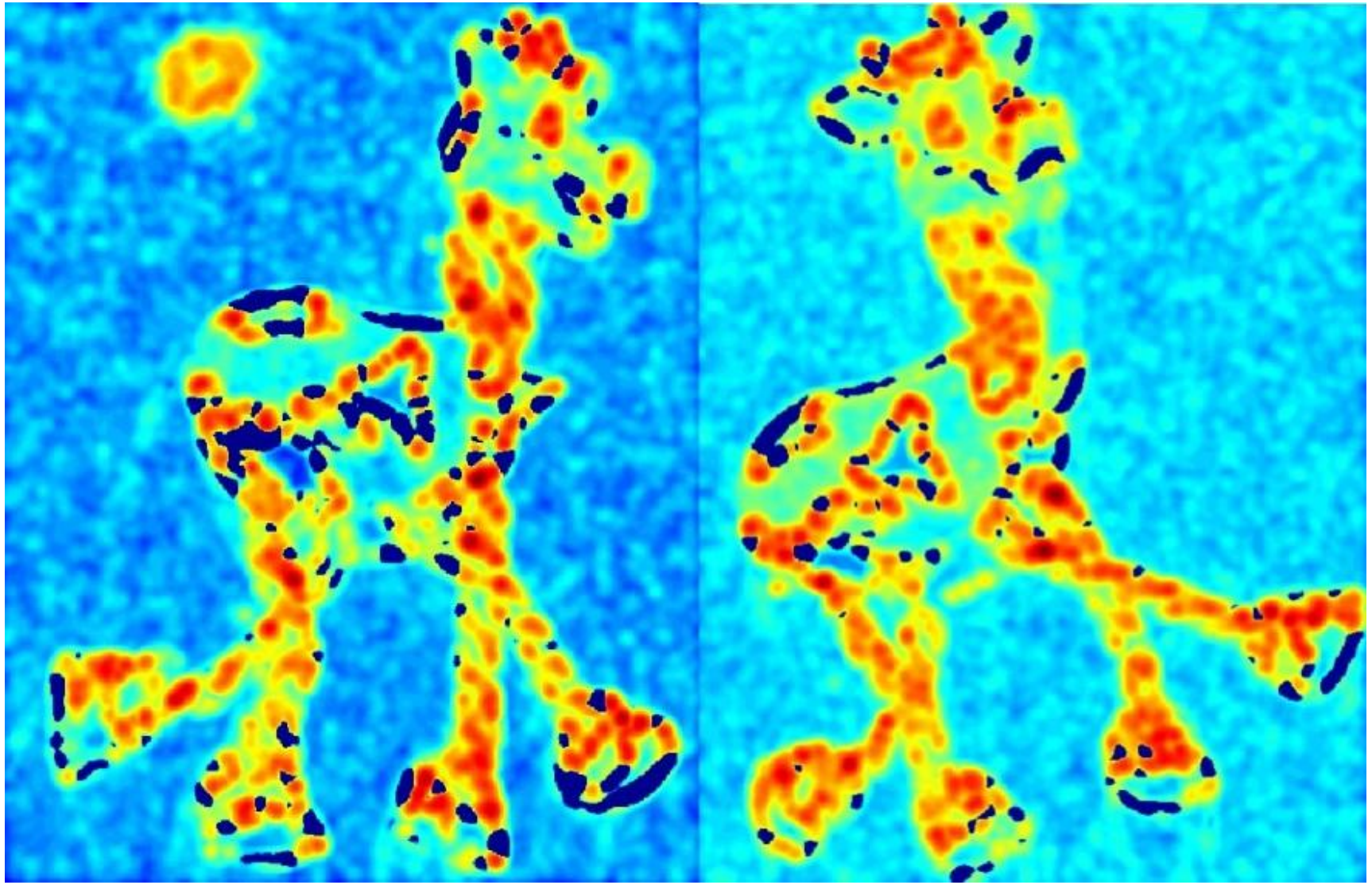
Harris Detector

- The Algorithm:
  - Find points with large corner response function R ($R >$ threshold)
  - Take the points of local maxima of R

Harris Detector: Workflow

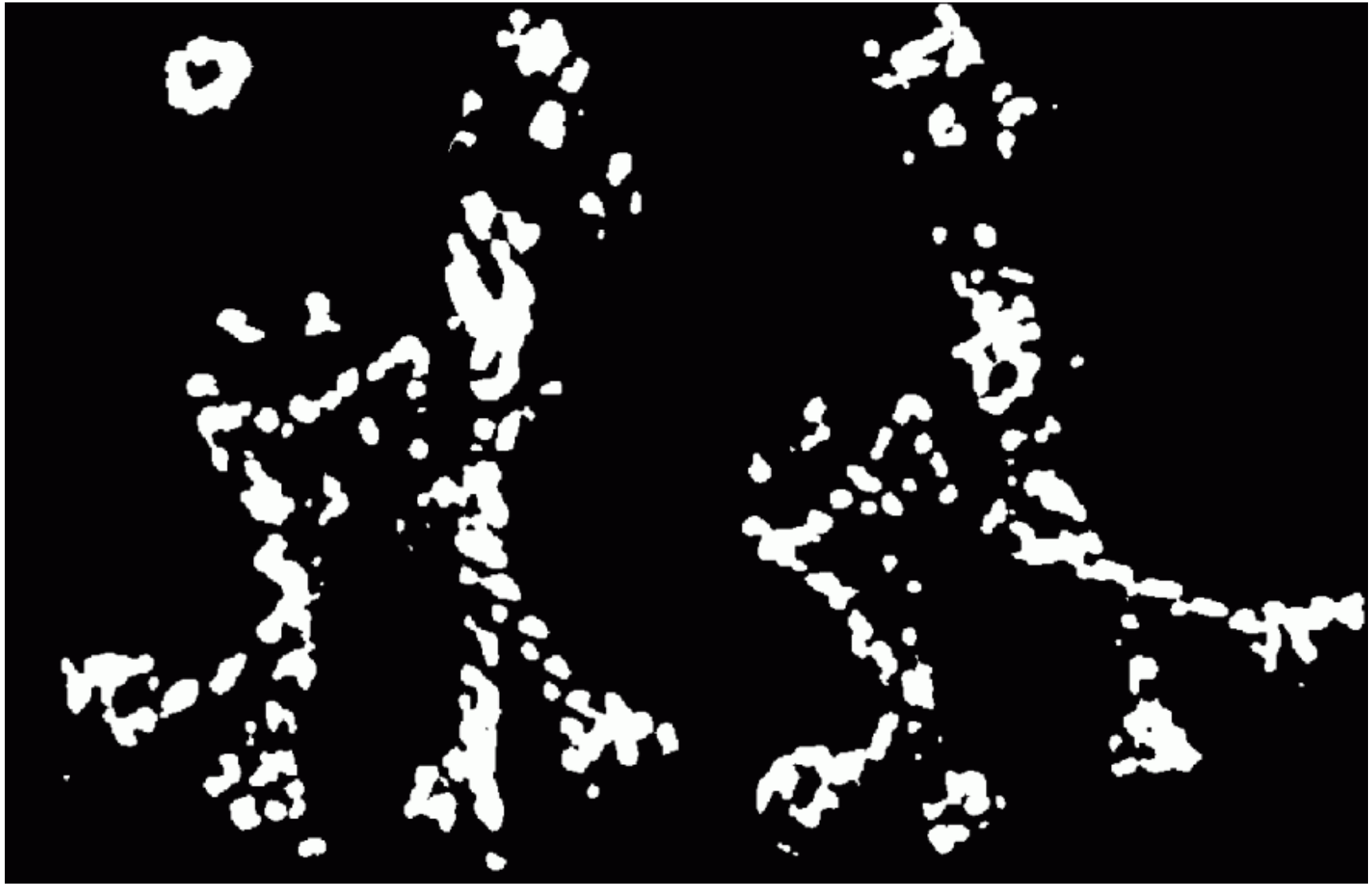
Harris Detector: Workflow

Compute corner response R



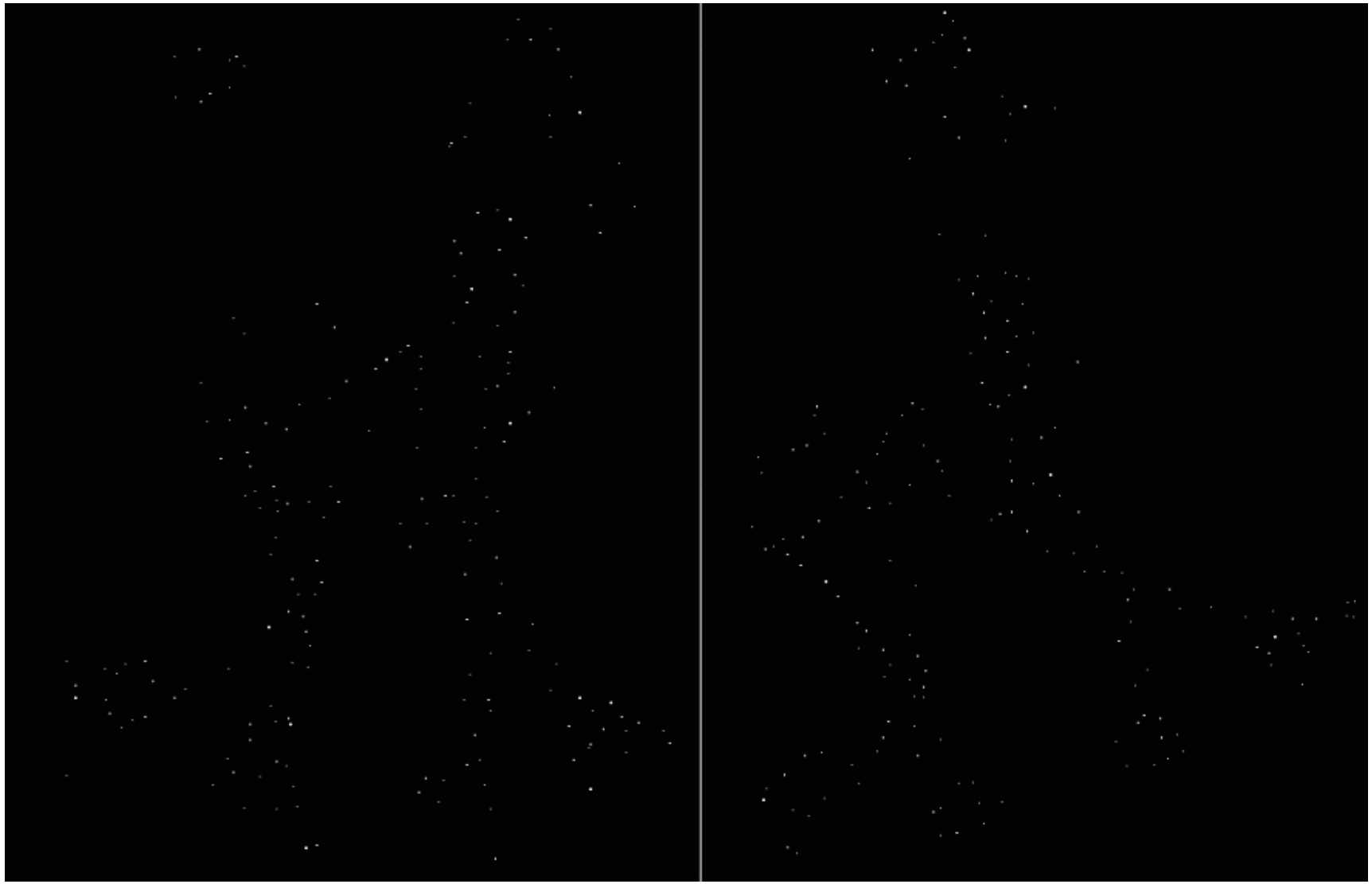
Harris Detector: Workflow

Find points with large corner response: R>threshold



Harris Detector: Workflow

Take only the points of local maxima of R



Harris Detector: Workflow

Algorithm 5.5: Harris corner detector

1. Filter the image with a Gaussian.
2. Estimate intensity gradient in two perpendicular directions for each pixel, $\frac{\partial f(x,y)}{\partial x}$, $\frac{\partial f(x,y)}{\partial y}$. This is performed by twice using a 1D convolution with the kernel approximating the derivative.
3. For each pixel and a given neighborhood window:
   - Calculate the local structure matrix A.
   - Evaluate the response function $R(A)$.
4. Choose the best candidates for corners by selecting a threshold on the response function $R(A)$ and perform non-maximal suppression.

Harris Corner Detector- Numericals

We are working with a 4×4 image matrix:

$$I = \begin{bmatrix} 100 & 120 & 130 & 110 \\ 150 & 180 & 190 & 160 \\ 120 & 140 & 160 & 130 \\ 80 & 100 & 110 & 90 \end{bmatrix}$$

Gradient Calculation (using central difference):

1. **Gradient in the x-direction (I_x)**:
   We calculate the gradient by considering the difference between adjacent pixels in the horizontal direction (left and right neighbors). After applying the gradient calculation, the resulting gradient matrix for I_x will have **4 rows and 3 columns** (because each row of the gradient will result in a value for the 3 interior pixels, as the first and last pixels do not have left and right neighbors, respectively).

   Example of I_x for a row (using central difference):

   For row 1:

$$I_x(1,1) = \frac{I(1,2) - I(1,1)}{2} = \frac{120 - 100}{2} = 10$$

$$I_x(1,2) = \frac{I(1,3) - I(1,1)}{2} = \frac{130 - 100}{2} = 15$$

$$I_x(1,3) = \frac{I(1,4) - I(1,2)}{2} = \frac{110 - 120}{2} = -5$$

Harris Corner Detector- Numericals

Gradient Matrices

Gradient I_x (Size: 4×3):

$$I_x = \begin{bmatrix} 10 & 15 & -5 \\ 15 & 15 & -5 \\ 10 & 15 & -5 \\ 10 & 15 & -5 \end{bmatrix}$$

Gradient I_y (Size: 3×4):

$$I_y = \begin{bmatrix} 25 & 30 & 30 & 25 \\ 10 & 10 & 15 & 10 \\ -35 & -40 & -50 & -35 \end{bmatrix}$$

Harris Corner Detector- Numericals

Step 2: Compute I_x^2, I_y^2, and $I_x I_y$

Now, we compute the squared gradients and the product of gradients to form the components of the **structure tensor** M.

I_x^2 (Element-wise square of I_x):

$$I_x^2 = \begin{bmatrix} 10^2 & 15^2 & (-5)^2 \\ 15^2 & 15^2 & (-5)^2 \\ 10^2 & 15^2 & (-5)^2 \\ 10^2 & 15^2 & (-5)^2 \end{bmatrix} = \begin{bmatrix} 100 & 225 & 25 \\ 225 & 225 & 25 \\ 100 & 225 & 25 \\ 100 & 225 & 25 \end{bmatrix}$$

I_y^2 (Element-wise square of I_y):

$$I_y^2 = \begin{bmatrix} 25^2 & 30^2 & 30^2 & 25^2 \\ 10^2 & 10^2 & 15^2 & 10^2 \\ (-35)^2 & (-40)^2 & (-50)^2 & (-35)^2 \end{bmatrix} = \begin{bmatrix} 625 & 900 & 900 & 625 \\ 100 & 100 & 225 & 100 \\ 1225 & 1600 & 2500 & 1225 \end{bmatrix}$$

$I_x I_y$ (Element-wise product of I_x and I_y):

$$I_x I_y = \begin{bmatrix} 10 \times 25 & 15 \times 30 & (-5) \times 30 \\ 15 \times 10 & 15 \times 10 & (-5) \times 15 \\ 10 \times (-35) & 15 \times (-40) & (-5) \times (-50) \\ 10 \times 25 & 15 \times 30 & (-5) \times 30 \end{bmatrix} = \begin{bmatrix} 250 & 450 & -150 \\ 150 & 150 & -75 \\ -350 & -600 & 250 \\ 250 & 450 & -150 \end{bmatrix}$$

Harris Corner Detector- Numericals

Step 3: Apply Gaussian Filter to I_x^2, I_y^2, and $I_x I_y$

Now we apply the **Gaussian filter** (denoted as G) to the components I_x^2, I_y^2, and $I_x I_y$ to smooth the values. Assuming a simple Gaussian filter, we get the following smoothed components (we'll use hypothetical values for simplicity):

$$G(I_x^2) = \begin{bmatrix} 150 & 200 & 100 \\ 200 & 200 & 100 \\ 150 & 200 & 100 \end{bmatrix}$$

$$G(I_y^2) = \begin{bmatrix} 700 & 900 & 850 \\ 150 & 150 & 225 \\ 1600 & 2200 & 2000 \end{bmatrix}$$

$$G(I_x I_y) = \begin{bmatrix} 300 & 400 & -100 \\ 150 & 150 & -75 \\ -350 & -500 & 200 \end{bmatrix}$$

Harris Corner Detector- Numericals

Step 4: Compute the Harris Corner Response R

Now we compute the **Harris corner response** using the formula:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

- $M = \begin{bmatrix} G(I_x^2) & G(I_x I_y) \\ G(I_x I_y) & G(I_y^2) \end{bmatrix}$
- $\det(M)$ is the determinant of the matrix M
- $\text{trace}(M)$ is the trace (sum of diagonal elements) of the matrix M
- k is a constant (typically $k = 0.04$)

Structure Tensor M:

$$M = \begin{bmatrix} 150 & 400 & -100 \\ 400 & 900 & 200 \\ 150 & 200 & 100 \end{bmatrix}$$

Determinant of M:

$$\det(M) = (150)(900) - (400)(400) = 135000 - 160000 = -25000$$

Trace of M:

$$\text{trace}(M) = 150 + 900 = 1050$$

Harris Corner Response R:

We use $k = 0.04$:

$$R = \det(M) - k(\text{trace}(M))^2$$

$$R = -25000 - 0.04 \times (1050)^2 = -25000 - 0.04 \times 1102500 = -25000 - 44100 = -69100$$

Harris Corner Detector- Numericals (contd..)

Step 5: Repeat for All Pixels

This process is repeated for all pixels in the image, and we get the Harris corner response for each pixel. **High values of** R indicate strong corners (areas with significant change in both x and y directions), and **low values of** R indicate flat areas.

Conclusion:

At the end of this process, you'll have a matrix R that contains the Harris corner response values for each pixel. You can then apply a threshold to find the locations of the strongest corners, or use a non-maximum suppression technique to extract the final corner points.

Histogram of Oriented Gradients (HOG)

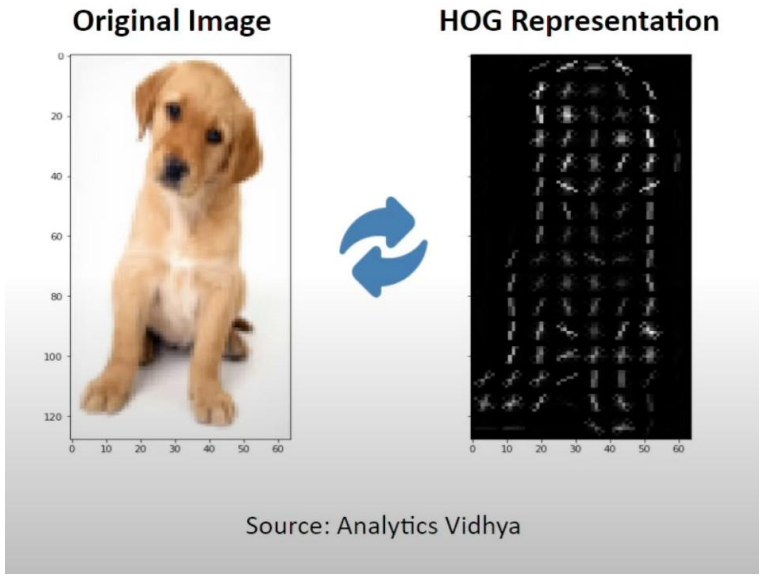
Histogram of Oriented Gradients (HOG) – An Introduction

What is HOG?

- **HOG** is a feature descriptor used in **image processing** and **computer vision** for object detection.
- It captures the **gradient orientation** (or edge direction) of local regions in an image, providing a representation that can be used to recognize patterns and objects.

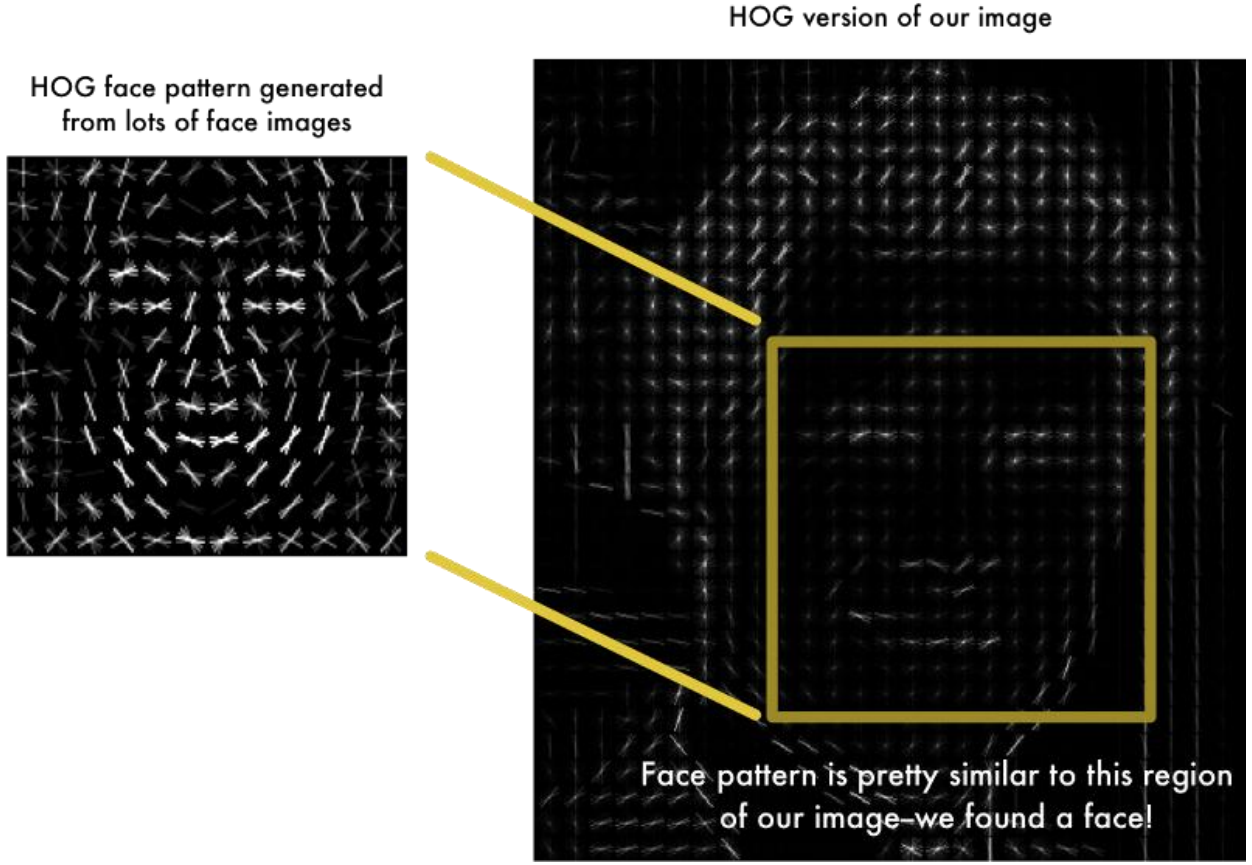
Feature	Label
F1, F2, F3,..., F352	Dog
F1, F2, F3,..., F352	??

https://www.youtube.com/watch?v=5nZGnYPyKLU



Applications of HOG

- **Feature Extraction:** from image data
- **Object Detection**: HOG features are commonly used in human detection (e.g., detecting pedestrians) and other object detection tasks.
- **Image Classification**: HOG can be used as input to classifiers such as **SVM** (Support Vector Machine) to recognize specific objects.
- **Face Recognition**: HOG features can aid in facial feature detection by capturing relevant edge and texture information.
- **Surveillance Systems**: Used in video surveillance for detecting objects, pedestrians, or vehicles.



Key Concepts

Gradient: Measures the intensity change in an image. The gradient direction indicates the edge direction.

Cells & Blocks:

- The image is divided into small **cells** (typically 8x8 pixels), and gradients are computed within these cells.
- These cells are grouped into **blocks** (typically 2x2 cells) for normalization, which helps reduce the effect of lighting variations.

Orientation Histograms: For each cell, a histogram of gradient directions is created, capturing the distribution of edge directions.

Steps

- Find horizontal and vertical gradients
- Gradient Magnitude and orientations
- Use a patch of 64 x 128
- Divide the image into blocks of 8 x 8 cells
- Slide over 2 x 2 block cells
- Quantize the gradient orientation into 9 bins by gradient magnitude
- Concatenate histograms into a feature of : 15 x 7 x 4 x 9 = 3780 dimensions.

Find horizontal and vertical gradients

Gradient Magnitude and orientations

$$\nabla f(x,y) = [gy] = {}^{g_x} \qquad \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \lim_{d \to 0} \frac{f(x+d) - f(x-d)}{2d}$$

Filter Masks:

-1	0	1

-1
0
1

Gradient Magnitude:

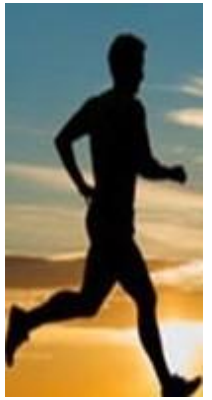
$$g = \sqrt{g^2 + g^2}$$

Gradient Orientation:

$$\theta = {}^{x}_{y} \tan^{-1}\left(\frac{g_y}{g_x}\right)$$

Blocks and Cells

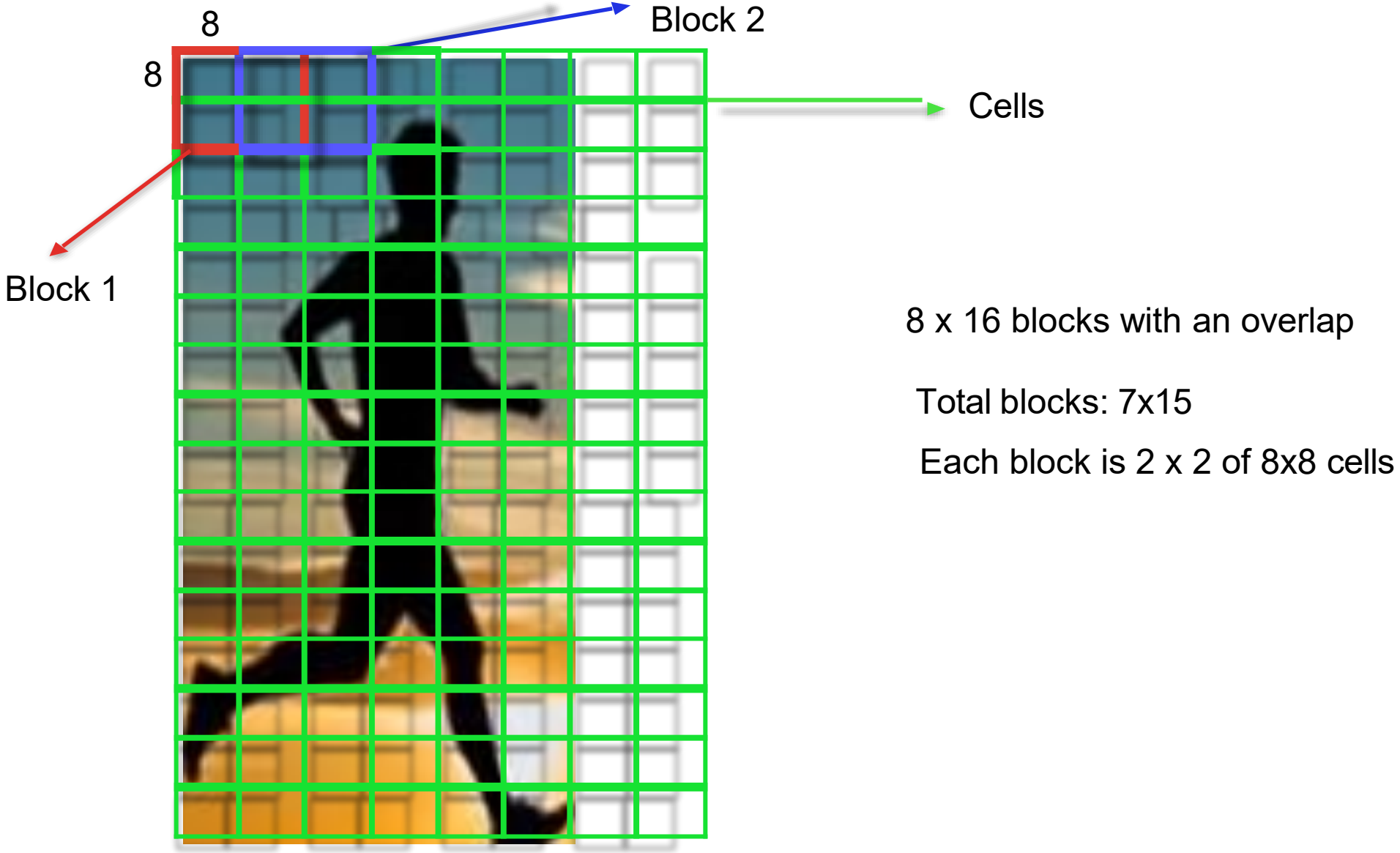
~~Use a patch of 64 x 128~~



128

64

Divide the image into blocks of 8 x 8 cells Slide over 2 x 2 block cells



8 x 16 blocks with an overlap

Total blocks: 7x15

Each block is 2 x 2 of 8x8 cells

Calculate Gradient and Orientation

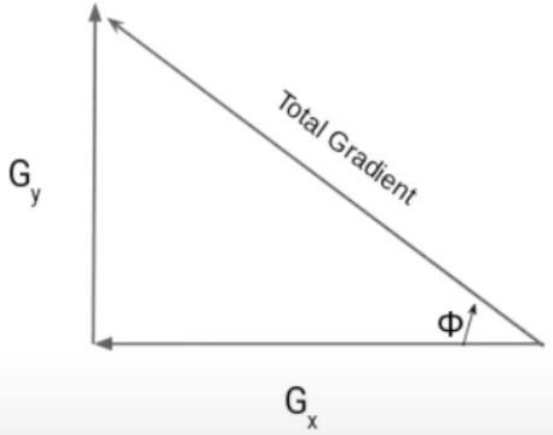
Select a 4 x 4 cell

Pixel values for selected cell

121	10	78	96
48	152	64	125
145	78	85	89
154	214	56	200

For edge pixels, HOG uses padding

From Pythagoras theorem



Gradient in x dir. (Gx)
= 89 - 78
= 11

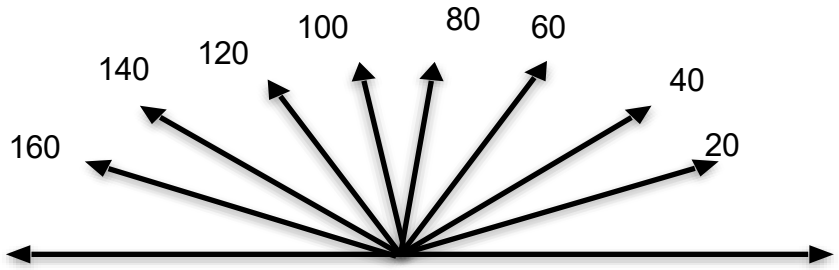
Gradient in y dir. (Gy)
= 64 - 56
= 8

Total Gradient Magnitude
$= \sqrt{[(Gx)^2+(Gy)^2]}$
$= \sqrt{[(11)^2+(8)^2]}$
= 13.6

Orientation (Φ)
= atan(Gy / Gx)
= 36 deg.

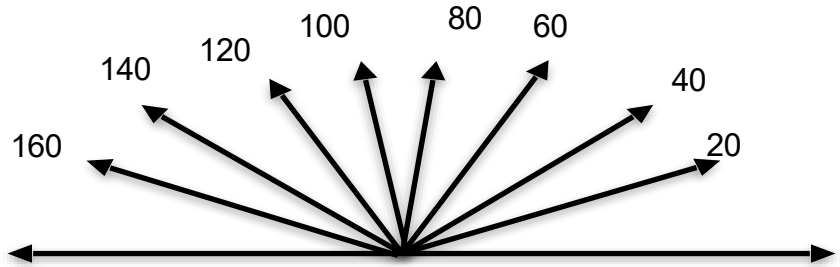
Histogram bins

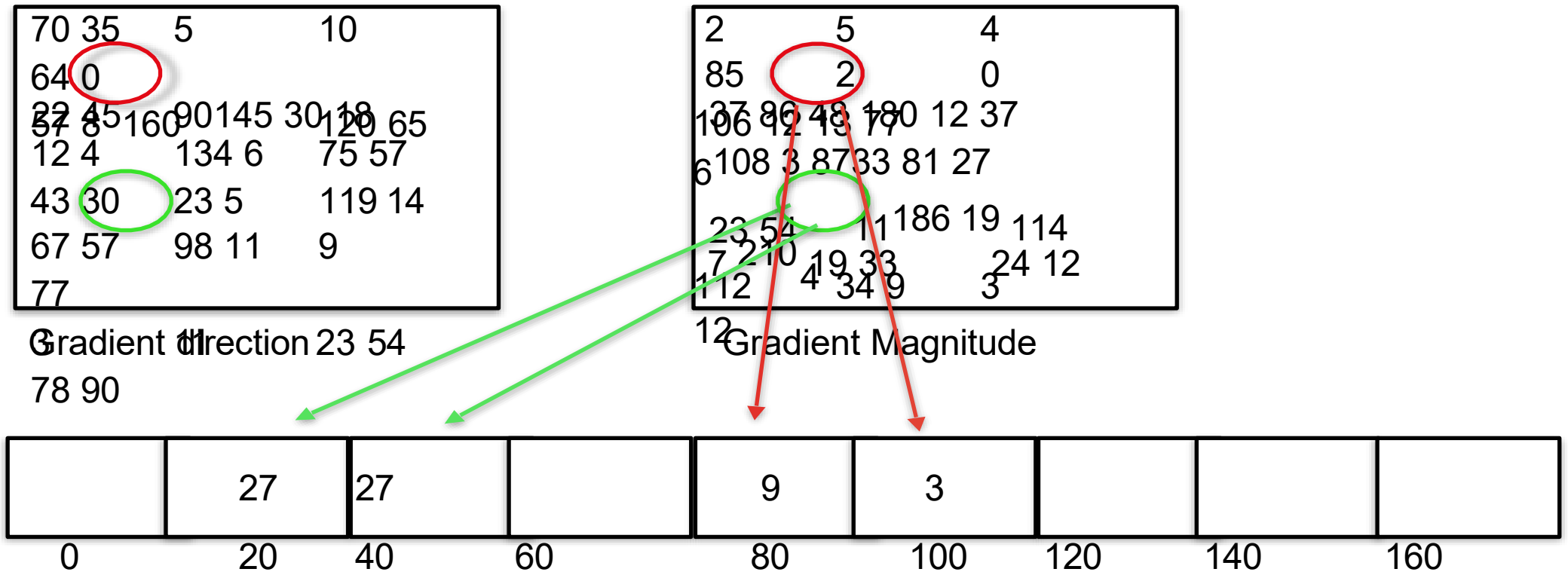
Quantize the gradient orientation into 9 bins by gradient magnitude



Histogram bins

Quantize gradient orientations into 9 bins





Gradient binning

Total Gradient Magnitude = 13.6

Orientation (Φ) = 36 deg.

(40-36)/20

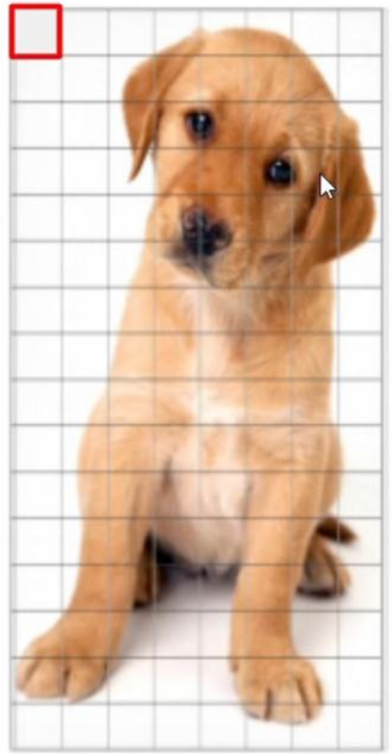
(36 - 20)/20

Magnitude		(4/20)*13.6	(16/20)*13.6						
Bin	0	20	40	60	80	100	120	140	160
Features	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

Weight

P32 | C8

Matrix 1 x 9



P64

C16

Normalisation

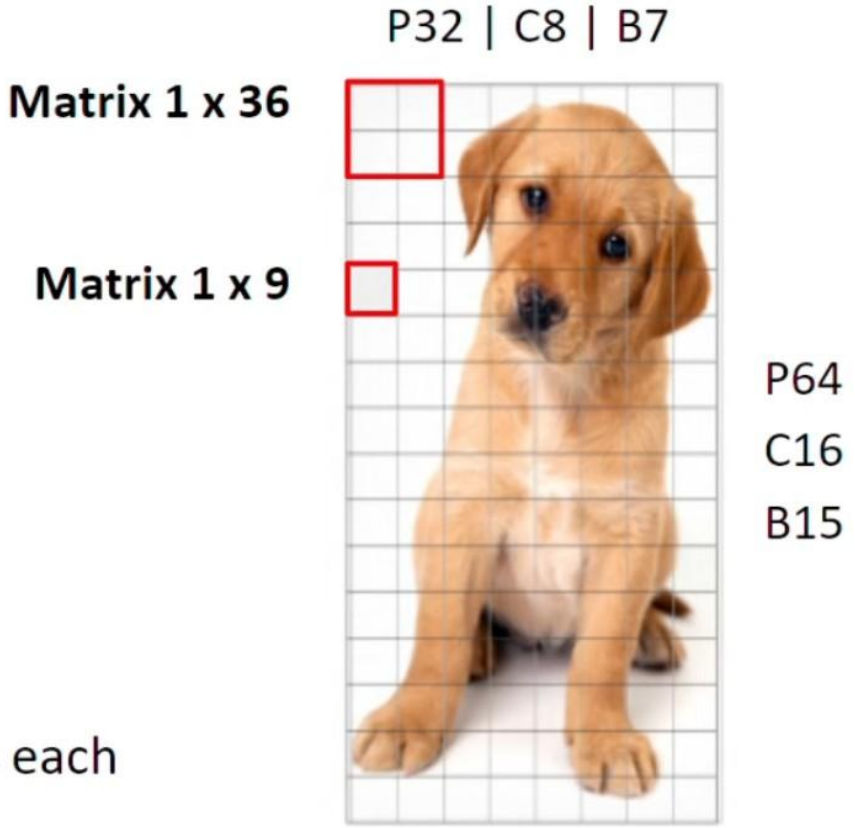
- Gradients for highlighted Block (having 36 features):

$$V = [a_1, a_2, a_3, \ldots a_{36}]$$

- Root of the sum of squares:

$$k = \sqrt{(a_1)^2 + (a_2)^2 + (a_3)^2 + \ldots (a_{36})^2}$$

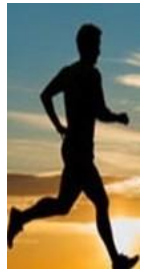
$$\text{Normalised Vector} = \left(\frac{a_1}{k}, \frac{a_2}{k}, \frac{a_3}{k}, \ldots \frac{a_{36}}{k} \right)$$

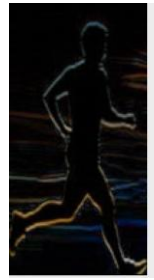


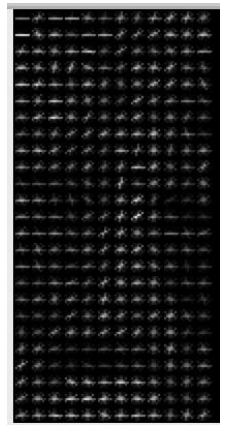
- Normalised vectors created for all 105 blocks, having 36 features each
- Image **Feature Descriptor: 1 x 3780 matrix**

Concatenate Histograms

15 x 7 * 9 * 4 = 3780







HOG Numericals

Let's consider a very small 5x5 image patch for simplicity. In a real scenario, HOG is applied to larger images and uses larger cells/blocks, but the core concepts remain the same.

Image Matrix (Grayscale):

```
[[80  70  50  70  90]
 [85  75  55  75  95]
 [90  85  60  80  100]
 [95  90  65  85  105]
 [100 95  70  90  110]]
```

Step 1: Gradient Computation

We'll use simple horizontal and vertical difference filters to approximate the gradients:

- **Gx (Horizontal Gradient Filter):** $[-1, 0, 1]$
- **Gy (Vertical Gradient Filter):** $[-1, 0, 1]^T$ (Transpose)

We apply these filters by performing a convolution. For simplicity, we'll ignore the edge pixels (in a real implementation, you'd typically pad the image).

Calculating Gx (Horizontal Gradient):

- For the element at (1,1) which is 75: $(55 * -1) + (75 * 1) = 20$
- We calculate this for the central 3x3 part of the image.

Gx Matrix (central part):

```
[[20  20  20]
 [25  20  25]
 [25  20  25]]
```

HOG Numericals Contd..

Calculating Gy (Vertical Gradient):

- For the element at (1,1) which is 75: $(70 * -1) + (85 * 1) = 15$
- We calculate this for the central 3x3 part of the image.

Gy Matrix (central part):

```
[[15  15  15]
 [10  10  10]
 [5   5   5]]
```

Gradient Magnitude and Direction

- **Magnitude:** $\text{sqrt}(Gx^2 + Gy^2)$
- **Direction (Angle in degrees):** $\text{atan2}(Gy, Gx) * (180 / pi)$ (atan2 handles all quadrants correctly)

Let's calculate these for the central 3x3 part:

Magnitude Matrix:

```
[[25.0  25.0  25.0]
 [26.9  22.4  26.9]
 [25.5  20.6  25.5]]
```

Angle Matrix (in degrees):

```
[[36.87  36.87  36.87]
 [21.80  26.57  21.80]
 [11.31  14.04  11.31]]
```

HOG Numericals Contd..

Step 2: Orientation Histograms in Cells

Let's assume our cell size is 3x3 (again, simplifying). We'll have one cell in this case, covering the central 3x3 part of our image.

We'll create a histogram with, say, 9 bins (0-20, 20-40, 40-60, ..., 160-180 degrees). For each pixel, we:

1. Find the bin corresponding to its angle.
2. Add its magnitude to that bin.

Creating the Histogram (for our single 3x3 cell):

- **Bin 1 (0-20):** 25.5 + 20.6 + 25.5 = 71.6 (from angles 11.31, 14.04, 11.31)
- **Bin 2 (20-40):** 25.0 + 25.0 + 25.0 + 26.9 + 22.4 + 26.9 = 151.2 (from angles 36.87, 36.87, 36.87, 21.8, 26.57, 21.8)
- **Bins 3-9 (40-180):** 0 (No angles fall in this range in our example)

Our 9-bin histogram (for the 3x3 cell):

[71.6, 151.2, 0, 0, 0, 0, 0, 0, 0]

Summary

This is a greatly simplified illustration of HOG. Here's a recap of the key differences in a full implementation:

- **Larger Images:** Real images are much larger.
- **More Cells and Blocks:** You'd divide the image into many cells and group cells into overlapping blocks.
- **More Histogram Bins:** 9 bins are common, but you can use more.
- **Proper Edge Handling:** Padding would be used to handle gradients at image edges.
- **Block Normalization:** Crucial for robustness.
- **Sliding Window:** HOG is often used with a sliding window to scan an entire image for object detection.

HOG Numericals Contd..

Step 3: Block Normalization

Since we only have one cell (due to our extreme simplification), we can't demonstrate block normalization here.

- **In a real scenario:** You would group cells into larger blocks (e.g., 2x2 cells).
- **Normalization:** You'd normalize the histogram of each cell within a block. Common methods are L1-norm, L2-norm, and L2-Hys. This makes the descriptor more robust to lighting changes.

Example of L2-norm (if we had a block):

1. **Concatenate** the histograms of all cells in the block.
2. **Calculate the L2-norm** of the concatenated vector: `sqrt(sum(element^2))`
3. **Divide each element** of the concatenated vector by the L2-norm.

Step 4: Final HOG Feature Vector

Again, since we have only one cell and no block normalization in this simplified example, our final HOG feature vector is simply the normalized histogram of our single cell.

Final HOG Vector (after hypothetical normalization):

Let's assume that after L2-normalization (which we can't perform with a single cell), the values in our histogram got scaled down. Our final vector could look like this (these are just example values after a hypothetical normalization):

`[0.45, 0.89, 0, 0, 0, 0, 0, 0, 0]`

Readings:

Histogram of Oriented Gradients:

(1) Histogram of Oriented Gradients https://courses.cs.duke.edu/fall15/cps274/notes/hog.pdf

(2) Histograms of Oriented Gradients for Human Detection Navneet Dalal and Bill Triggs https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf

Harris Corner Detector

(1) Milan Sonka , (Our Textbook-2) : Section 5.3.10

[optional, yet recommended reading , Milan Sonka Section 5.3.11]

Notebook

https://github.com/mithunkumarsr/LearnComputerVisionWithMithun/blob/main/CV6_HarrisCornerDetection_HOG.ipynb

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Thank you